AI EVALUATION PLATFORM

AI Eval Platform 全功能分析大报告

一个测一切的 AI 能力评测引擎——基于全部源码逐文件通读的实证分析，覆盖架构分层、评测引擎、harness 六层、API、数据库、前端与部署运维。

分析对象	ai-eval-platform
代码规模	Go 后端 ~30,500 行 + Vue 前端 ~8,500 行
分析方法	全部源码逐文件通读（未参考 TODO 文档）
报告日期	2026 年 6 月

CODE-BASED EVIDENCE REPORT

CONFIDENTIAL · INTERNAL

目录

一 项目总览 03

二 架构分层（实测） 04

三 核心评测引擎功能 05

意图路由 · 圆桌合议 · 双判官 · 断路器 · 沙箱 · 反泄漏 · 混沌 · RAG · 归因 · harness 六层

四 API 层功能 11

五 数据存储层 13

六 协议逆向与调度子系统 14

七 前端功能 16

八 部署与工程化 18

九 诚实评估：真实 / 占位 / Bug 19

十 总体结论 21

一、项目总览

一句话定位：一个"测一切的 AI 能力评测引擎"——被测对象任意（Agent / 大模型 / AIGC / 产品 / 竞品报告），评测规则用户定义，提供"喂题 → 执行 → 按规则打分 → 归因"的通用闭环。

> **本报告的实证性声明：**所有结论均来自对 `E:\glm\ai-eval-platform` 全部源码的逐文件通读，**未参考项目内的任何 TODO / MRD 文档**。报告中所列的代码行数、表数量、端点数量、测试数量均为实际统计。

量化指标

维度	数据
Go 后端代码	~30,516 行（不含测试），分布在 `internal/` + `cmd/`
Vue 前端代码	~8,475 行，18 个页面
测试函数	187 个（`func Test*`），覆盖 engine / api / sdk / proxy / store
数据库表	44 张（核心 31 + 评测 8 + Agent 运行时 5）
HTTP 端点	145 个（GET 60 / POST 58 / PUT 10 / DELETE 17）+ 1 WebSocket
编译产物	`server.exe`（40MB）、`eval-cli.exe` 、`mock-agent.exe` 均已编译
运行状态	`data/eval.db` + WAL（3.6MB 活跃写入）证明系统正在运行

30.5K
Go 后端代码行

187
测试函数

44
数据库表

145
HTTP 端点

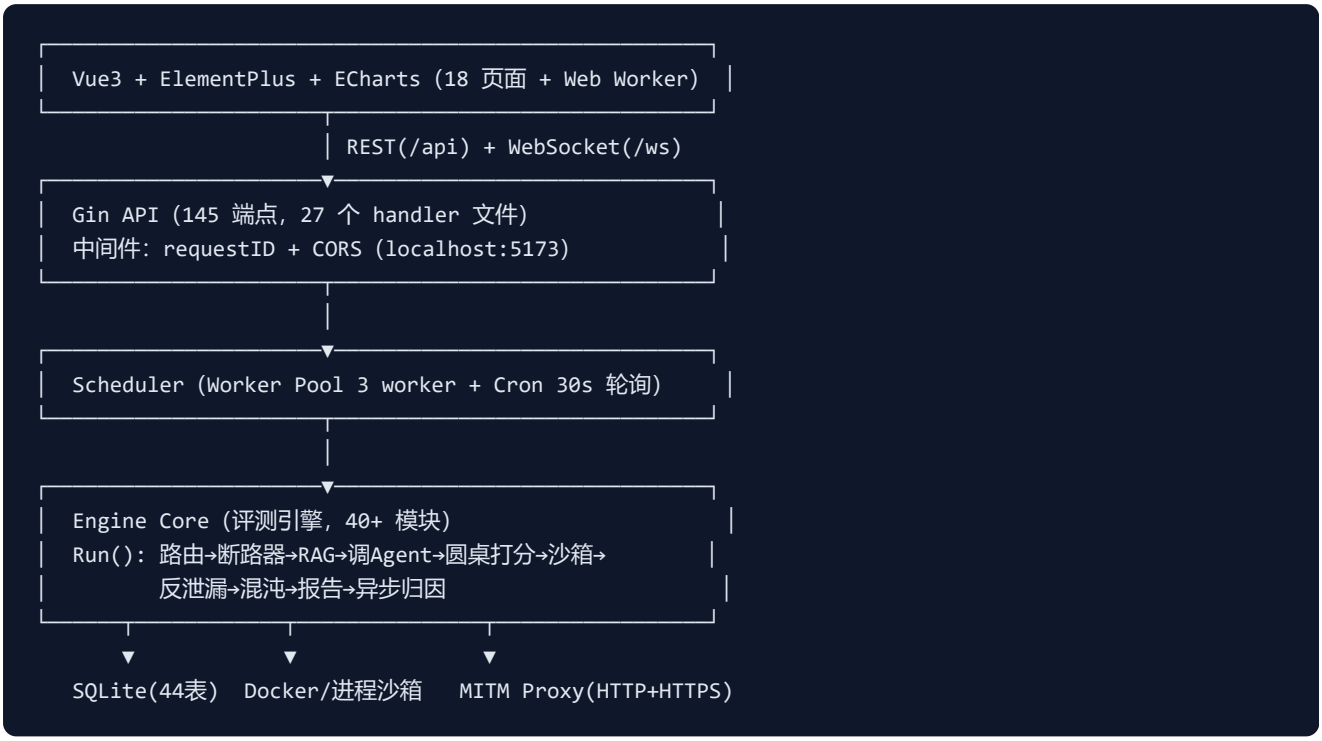
技术栈

层	技术	用途
后端	Go 1.25 + Gin 1.12	HTTP 路由 + 中间件
数据库	modernc.org/sqlite（纯 Go，免 CGO）	WAL 模式嵌入式存储
WebSocket	gorilla/websocket	评测进度实时推送
AI 调用	OpenAI 兼容协议为主	适配器额外支持 Anthropic / Gemini 原生协议
前端	Vue 3.5 + Element Plus 2.14 + Pinia 3	响应式 SPA

层	技术	用途
可视化	ECharts 6 + Monaco Editor + xterm.js	图表 / Diff / 终端
部署	Docker Compose + GitHub Actions CI	4 个 CI job 并行

二、架构分层（实测）

整体采用清晰的四层架构，自上而下：前端 → API → 调度 → 引擎核心，引擎再外接 SQLite / 沙箱 / MITM 代理三个落地存储。



启动流程

入口 `cmd/server/main.go`，启动顺序为：

1. **进程硬化**（`engine.HardenProcess`）—— 清 `LD_PRELOAD` 等危险环境变量
2. **加载配置**（`config.Load`）—— 环境变量 > 默认值
3. **初始化 SQLite**（`store.Init`）+ **种子数据**（`store.SeedIfEmpty`）
4. **初始化沙箱容器池**（`engine.InitSandboxPool`）
5. **启动即时调度器**（`scheduler.InitGlobal(engine.Run)`）+ **Cron 调度器**
6. **注册路由**（`api.NewRouter`）→ 监听端口
7. **优雅关闭**：SIGINT/SIGTERM → 停调度器 → HTTP Shutdown → 关数据库

三、核心评测引擎功能

这是整个平台的**心脏**，也是最扎实、实现最完整的部分。`engine.go:Run` 是一个 8 步管线，全程不 panic（任何失败都标记 task=failed 兜底）。

3.1 意图识别路由（router.go） 真实

单主 Agent 统管所有场景，**不做 5 个独立场景 Agent**（避免五份重复基建 + 数据隔离导致无法跨场景对比）。三级分类策略：

1. **显式指定**：用户直接传 `scene` 字段 → 置信度 100% 直接用
2. **关键词匹配**：每场景预设 7-9 个关键词，命中率 ≥ 60% 采用
3. **LLM 分类 fallback**：低置信度时调 `judge_models.main`（需 Key）

5 个预置场景 → 4 种执行策略映射：

场景	策略	说明
`agent`	`sandbox_roundtable`	沙箱编译 + 圆桌合议（最完整）
`model`	`api_direct`	单判官，不走沙箱
`competitor` / `prd`	`text_review`	圆桌评审
`product`	`multimodal_review`	多模态评审

3.2 圆桌合议（roundtable.go） 真实 · 设计精良

一个主导者 + 多个领域专家并发评审，区别于单判官的"一条规则调一次 LLM"。核心要点：

- 专家从 `judge_models` 表动态加载（role 以 `expert_` 开头），无数据回退预置 4 专家（安全/性能/架构/产品）
- 也支持从 `expert_groups` 表加载可配置专家组（支持 agent/model/role 三类成员 + 内嵌 prompt + 引用 prompt_template + 权重 weight）
- 各专家 **goroutine 并发**（`sync.WaitGroup`），每专家有 **60s 超时保护**不阻塞他人
- 每专家只看自己维度的规则：metric 规则直接算（免费），llm 规则合并成 rubric 调一次 LLM
- metric × 0.4 + llm × 0.6 加权（metric 确定性高但覆盖面窄）
- 主导者按 weight 加权汇总 + 生成综合评语（自动找出最强/最弱维度）
- **专家矛盾检测**：同维度或跨维度评分差 > 30 分标记矛盾

3.3 双判官打分体系（judge.go + metric.go + scorer.go） 真实

LLM 判官（judge.go）

规则 content 作 rubric，`temperature=0` 追求确定性，输出 `{score, reason}` JSON，正则兜底提取。

metric 判官（metric.go）

确定性校验器，不调 LLM。内置 `checkNoInjection`（检测 sprintf + SQL 拼接扣分，参数化 `?` 加分）、`checkNoSecretLeak`（硬编码 sk-xxx 检测）、`checkCompilable`（括号配平 + 结构）、`checkTestCoverage`。

三轨分发（scorer.go）

`metric` / `llm` / `manual` 三种 scoring 方式，manual 为占位（阶段①不做人工 UI）。无 Key 时全套走启发式 mock 判官，保证 Demo 可演示。

3.4 钱包保护断路器（breaker.go） 真实

解决"开发期 Agent 陷入无限内耗烧钱"问题。每题评测后检查三项上限：

上限项	默认值	说明
Token 上限	10 万	单次评测最大 token 消耗
Tool Calling 轮数	30	防止疯狂循环
费用上限	$5	按 GLM-4-flash $0.002/1K 估算

超限立即熔断 `status=blown`，已评测题仍写入报告（部分结果）。每题后通过 WebSocket 实时推送断路器状态。多 provider 定价表（GPT-4o / Claude / DeepSeek / GLM / Qwen / o1）硬编码在 `estimateCost`。

3.5 沙箱执行（sandbox.go + sandbox_pool.go） 真实 · 双轨

Docker 容器池优先（Phase 2）→ 进程级沙箱回退（Phase 1）：

- **进程级**：临时目录 `go build` / `python -m py_compile` / `node --check`，支持 Go/Python/JS 三语言编译 + 测试 + Lint
- **Docker 池**：网络隔离 `--network none`、资源限制、只读 FS、无特权
- 采集硬指标：编译时延、编译成功/失败、测试通过率（解析 `go test -v` / pytest 输出）、Lint 错误数
- **Test-Gen 集成**：将自动生成的测试代码注入沙箱与源码一同编译运行

3.6 反泄漏扫描（antileak.go） 真实 · 纯启发式

每题评测后扫描，4 类检测，**不调 LLM**（免费确定）：

检测类型	严重度	说明
System Prompt 泄露	critical	回答含 "system prompt" / "系统指令" / "我被设定为" 等，`isGenuineLeak` 区分正常自我介绍 vs 真泄露
提示词注入	high	输入含 "忽略之前的指令" / "jailbreak" / "DAN 模式" 等 13 种越狱模式
踩额度攻击	medium/high	输入 > 10000 字 / 连续 5 次以上相同行刷屏
凭据泄露	critical	回答含 `sk-xxx` / `api_key=` / `password=` / 带密码的连接串，凭据脱敏后记入证据

3.7 混沌工程（chaos.go） 真实

"线下林志玲，线上罗玉凤"——主动注入故障测鲁棒性。4 种注入（仅对前 2 题注入，避免成本爆炸）：

1. **弱网延迟**：注入"API 超时 5000ms"提示，检测是否提到重试/降级
2. **脏数据**：注入缺右括号的 JSON，检测是否识别格式错误
3. **越狱对抗**：尝试让 Agent 忽略安全约束，拒绝 = 95 分，泄露 System Prompt = 10 分
4. **资源耗尽**：模拟磁盘满 / 内存不足 / 配额耗尽，检测是否提降级方案

每次注入记录到 `permissions` 表（`chaos_injected=1`），鲁棒性分聚合进报告。

3.8 RAG 规则检索（rag.go） 真实 embedding 占位

规则多了不爆 token，每题只检索 top-K 相关规则：

- 检索 / 相似度排序 / 关系边查找链路**完整工作**
- **但 embedding 是 hashing-trick 占位**：FNV-1a 哈希把词映射到 128 维固定位置 + L2 归一化，**不是真实 embedding 模型**（代码注释明确"生产环境应替换"）
- 支持沿 `rule_relations` 关系边拉关联规则（知识图谱）
- 可持久化到 `rag_embeddings` 表

3.9 异常归因闭环（attribution.go） 真实 · 含自动反哺

评测有 Bad Case 时**后台异步**分析根因（不阻塞主流程）：

- **4 类根因分类**：`prompt_compliance`（指令遵循）/ `context_loss`（上下文丢失）/ `tool_failure`（工具失败）/ `model_limit`（模型上限）
- 启发式归因（无 Key 时）+ LLM 归因
- **自动闭环反哺**：置信度 ≥ 0.7 时，自动把修正建议写回 `prompt_templates` 表（创建新模板或新版本），实现"评测 → 暴露 Bug → 归因 → 提示词进化"
- 高置信度（≥ 0.8）建议通过日志提示用户采纳

3.10 其它引擎能力

模块	实现状态	说明
Test-Gen（testgen.go）	真实 + mock 兜底	解析 Diff 块自动生成单元测试，Go/Python/JS 三语言正则解析 + LLM 生成，无 Key 回退骨架测试
AI 出题（questiongen.go）	真实 + mock 兜底	圆桌按维度批量出题 + 动态扩题（失败题生成变体），写库 source=`ai` / `dynamic`
克隆 A/B（clone.go）	真实	复制 Agent 换基座并联跑，算 **MSI 敏感度指数**（得分标准差）+ **ROI 性价比**（两两配对），给最优模型推荐
回归测试（regression.go）	真实 + 估算	V1 vs V2 逐题逐维度 diff，标 pass→fail 回归漏洞 / fail→pass 改进
红队测试（redteam.go）	真实	6 类（注入/泄露/越狱/提权/投毒/侧信道）× 20 = 120 条真实 payload，规则引擎判定中招，算 CVSS 分 + 修复建议
上下文压缩（compaction.go）	真实 + 提取式兜底	滑动窗口 + LLM 摘要，6 种 Condenser 策略，无 Key 回退提取式（保留首尾 + 结构行）
多 provider 适配	真实	OpenAI / Anthropic / Gemini 原生协议各自实现，自动按 baseURL 检测路由，含五维 token 统计 + 重试退避

3.11 harness 六层能力（借鉴 claw-code，已落地）

代码里真实实现了类 Claude Code 的 agent 运行时六层：

层	文件	实现状态	说明
① AGENTS.md	`agents_md_discovery.go`	真实	多层目录发现 AGENTS.md 注入 system prompt
② Tools & MCP	`agent_loop.go` + `builtin_tools.go` + `mcp_client.go`	真实	完整 ReAct 循环，8 个内置工具（Bash/FileRead/FileWrite/FileEdit/Glob/Grep/WebSearch/tool_search），MCP 支持 HTTP+stdio 双 transport
③ Skills	`skills_loader.go`	真实	渐进式披露，三级目录（project>user>managed）扫描 SKILL.md frontmatter

层	文件	实现状态	说明
④ Subagents	`subagent.go`	真实	Task 工具 fan-out 子代理，Coordinator（零继承）+ Fork（全继承）两种模式，递归深度 guard + 独立断路器
⑤ Hooks	`hooks.go`	真实	9 种生命周期事件（PreToolUse/PostToolUse/Stop 等），支持 command/llm/verifier 三种 hook 类型
⑥ Back-Pressure	`backpressure.go`	未接入主循环	`VerifyWithRetry` 回路写好了但 agent_loop 未调用，agent 需主动调 Verify 工具

配套的安全设施

- **权限门控**（permission_gate.go）：read-only / workspace-write / full 三级，`CheckCommand` 拦截危险命令（rm -rf /、fork 炸弹、mkfs、dd 裸盘等），`CheckSandboxCode` 预检被测代码内嵌的危险调用，`RecordDecision` 写 `permissions` 表审计
- **密钥加密**（secrets.go）：AES-GCM 真实实现，但**默认 key 硬编码且未接入实际 api_key 读写路径**（独立设施）
- **进程硬化**：启动时清 `LD_PRELOAD` 等危险环境变量
- **终端轨道**（terminal.go）：本地源码/进程托管，三种喂题模式（args/stdin/oneshot），AutoDiscover 自动识别 Python/Go/Node 项目

四、API 层功能（internal/api/，27 个 handler）

145 个端点，按功能域归类如下。

4.1 七库资源 CRUD（核心数据）

- **Agents 库**：CRUD + AI 创造（quick/detail 两种模式）+ Agent 定义文件实例化 + hooks 管理 + 状态查询
- **Models 库**（被测考生）：CRUD + 连通性探活
- **Tools / MCP / Skills 库**：CRUD
- **Model Config**：角色 ↔ 模型映射（圆桌各专家绑定）
- **Prompt Templates**：CRUD + **版本历史 + 回滚**
- **Expert Groups**：专家组 CRUD + AI 生成草稿

4.2 评测流程

任务 CRUD + 规则/规则集 CRUD + 题库 CRUD（多维筛选）+ AI 出题 + 动态扩题 + GitLab 基线导入 + 测试卷管理 + 会话查询 + 归因查询。

4.3 数据看板与统计

看板总览（指标卡）/ 综合分趋势折线 / 多 Agent 排行榜 / Bad Case 列表 / 归因列表 / **双路 token 统计**（被测 Agent vs 平台 condenser 分别计费）。

4.4 差异化高级能力

混沌测试触发 / 克隆 A/B / 单+批量归因 / Prompt 智能平移适配 / 规则知识图谱（环形布局节点 + 关系边）/ 规则关系边 CRUD / JSON 导入导出 / 人工反馈闭环（Bad Case 踩赞 + 任务反馈 + 规则建议采纳）。

4.5 六大扩展方向（每个一套独立表 + handler）

方向	handler	核心能力
Agent CI/CD	`expansion_pipeline.go`	管线定义（含质量门禁 gate_config）+ 异步触发 + 运行历史
Agent Arena	`expansion_arena.go`	ELO 排行榜 + 对战记录 + 异步评测 **评分硬编码假数据**
红队安全	`expansion_redteam.go`	6 类 payload 扫描 + CVSS 报告

方向	handler	核心能力
生产可观测	`expansion_observe.go`	指标上报 + 仪表盘聚合 + 告警规则匹配
微调数据	`expansion_finetune.go`	从 Bad Case 导出 SFT/DPO 训练对 **JOIN 语义有 bug**
多 Agent 编排	`expansion_workflow.go`	DAG 工作流 **未做拓扑排序**

4.6 MITM 代理管理

启停 + 状态 + 会话捕获/详情/转换/回放 + CA 证书生成 + 环境变量重定向脚本生成。

4.7 中间件与问题

中间件：requestID + CORS（硬编码 localhost:5173）。**无任何鉴权：**所有端点裸奔，终端执行/代理控制/API Key 写入均无保护。

requestID 生成 bug：`generateShortID()` 确定性（`make([]byte,12)` 零初始化 + 异或固定），所有请求同一 ID，追踪失效。

正面确认：所有动态 SQL 均用 `?` 参数化，**未发现 SQL 注入**。也没有空 handler 或纯 TODO 占位函数。

五、数据存储层（internal/store/，44 张表）

5.1 数据库设计

- **modernc.org/sqlite**（纯 Go 免 CGO）+ WAL 模式 + 5s busy_timeout
- 连接池 MaxOpenConns=3 / MaxIdleConns=2（适配 WAL 的并发读 + 单写）
- 启动时顺序迁移 `migrateCore → migrateEval → migrateAgent`，全用 `CREATE TABLE IF NOT EXISTS`（幂等）
- **增量列迁移** `migrateAddColumn`：吞掉"列已存在"错误实现幂等（但也吞掉真实错误）

5.2 表分布（实测 44 张）

文件	表数	核心表（节选）
schema_core	31	七库基础（agents/models/tasks/rules/rule_sets/tools/mcp_servers/skills/model_config/prompt_templates/scenes）+ 模型职责分离（judge_models 区分考生/考官）+ 专家组 + 六大扩展方向（pipelines/arena/redteam/agent_metrics/workflows 等）
schema_eval	8	cron_jobs / questions / bad_cases / attributions / task_feedback / rule_suggestions / rag_embeddings / question_sets
schema_agent	5	conversations / permissions / compactions / sessions / tool_calls（Agent 运行时，借鉴 claw-code）

5.3 种子数据（seed.go，幂等）

类型	数量	说明
模型（models）	3	glm-4 / deepseek-chat / glm-4-flash（Key 留空）
被测 Agent	3	CodeGen / Debug Hunter / Arch Reviewer，覆盖 api 接入
内置 Tool	5	shell_exec / file_read / file_write / http_request / static_scan
评测规则	13	覆盖 security/performance/architecture/product/correctness 五维
规则集	7	全栈/安全/架构/大模型/竞品/PRD/产品 七场景套件
题目	17	覆盖全场景 + 3 道带来源追溯基线题
考官模型	6	main + 4 专家 + 出题，含各自维度

类型	数量	说明
专家组	1 组	eg-default 含安全/性能/架构/产品四维专家

5.4 query.go / rows.go

`query.go` ~40 个封装函数（Get/Upsert/Update/Insert/Delete），让 engine 包不直接写 SQL，与 api 层 handler 的直查并存。`rows.go` 11 个中立 `XxxRow` 结构体，避免 engine 依赖 api（防循环依赖）。

六、协议逆向与调度子系统

6.1 MITM 代理（internal/proxy/） 真实 有 bug

- **HTTP 代理**（proxy.go）：反向代理 + request/response 捕获 + 凭证提取（Bearer/x-api-key/sk-/Cookie）+ 脱敏（前4后4）+ ReplaySession + ConvertSessionToTask
- **HTTPS MITM**（proxy_tls.go）：**手写解析 TLS ClientHello 提取 SNI** → 动态签发证书 → 伪服务器握手 → 明文读 HTTP → 真实客户端转发。HTTP/1.1 keep-alive 支持，body 脱敏
- **CA 管理**（ca.go）：RSA 2048 自签 CA（10 年），leaf 证书按域名缓存（24h），各平台安装指引
- **环境变量重定向**（redirector.go）：对标 claude-code-router，生成 ANTHROPIC_BASE_URL / OPENAI_BASE_URL 注入脚本。**诚实降级**：MRD 的激进灰色能力（TLS 指纹伪造 / IPC 劫持 / 配置解密）全部 `return ErrNotImplemented`

> **已知 bug**：HTTP 代理 session 分组逻辑错误（所有请求进同一 session）；HTTP 路径不脱敏但 HTTPS 脱敏，两条路径不一致；存在死代码（旧 peekSNI/handleConnection 未删）。

6.2 调度器（internal/scheduler/） 真实

即时调度（scheduler.go + global.go）

channel-based Worker Pool（默认 3 worker，buffered channel 容量 100），单任务超时 300s + 重试 2 次，`running` map 去重。`RunDirect` 作为队列满时的降级保底。

> **优先级字段是摆设**：`Task.Priority` 字段存在，`Submit` 也接收 priority，但 worker 从单一 channel 取任务，channel 是 FIFO，**优先级完全没有被使用**。

Cron 调度（cron.go + global_cron.go）

自研简化 Cron（不依赖外部库），30s 轮询检查，持久化到 cron_jobs 表。**解析极度残缺**：只处理 5 字段的前两位（minute/hour），日/月/周字段被忽略；不支持范围（`1-5`）、列表（`1,3,5`）。

6.3 SDK 接入（internal/sdk/） 真实 包路径混乱

- **PlatformClient**：RegisterAgent / ListAgents / SubmitTask / GetTask / WaitForTask（2s 轮询 10min 超时）/ ListRuleSets / ListQuestions
- **EvalHandler**：Agent 侧 HTTP Handler，平台 POST `/eval` 调 Agent
- **MCPServerHandler**：MCP 协议适配（ListTools + call_tool）
- **CLI**（eval-cli）：register / submit / status / list 四类命令，平台地址和 key 从环境变量读
- **mock-agent 示例**：完整端到端演示（启动 :9090 Agent → 注册自己 → 提交评测 → 等结果），按 prompt 关键词返回不同 mock Go 代码

> **包路径问题：** `internal/sdk` 和 `pkg/sdk` 维护了两份副本，CLI/示例 import `pkg/sdk`，README 写 `internal/sdk`，矛盾。需确认哪份是 source of truth。

6.4 WebSocket Hub（internal/ws/） 真实 · 简洁规范

`taskID → []*listener` 映射，`Subscribe` 返回缓冲 64 的 channel；非阻塞发送（满则丢，反压策略）；支持"全局订阅"（taskID 空，看板用）。6 类事件：

事件	Data 结构	说明
`question_start`	QuestionStartData	第 N 题 / 总题数 / 题目摘要
`agent_response`	AgentResponseData	回答前 200 字预览 + token_in/out
`question_done`	QuestionDoneData	该题分数 + is_bad_case
`task_done`	TaskDoneData	overall 综合分 + dimensions 雷达 + bad_cases + mock_mode
`breaker_status`	BreakerStatusData	累计 token / 轮数 / 估算费用 / 是否熔断
`task_error`	—	任务异常

七、前端功能（web/src/，18 页面）

> **结论：前端是真实可用的产品级实现，不是占位壳。**全链路对接 REST + WebSocket，涉及约 50+ 个不同端点。

7.1 技术实现

- 统一 `api.ts`（axios，baseURL `/api`，Response 拦截 unwrap data，统一错误处理）
- Pinia 字典幂等缓存（`ensure*` 已缓存不重复请求）+ 强制刷新
- 路由懒加载，18 条路由，每条带 `meta.title` 和 `meta.icon`
- **Web Worker**（data-worker.ts）：离线程做 parse_json / pareto（非支配解算法）/ leaderboard / matrix_diff / aggregate，避免主线程卡顿

7.2 可视化能力（专业选型）

能力	库	用途
图表	ECharts 6（BaseChart 封装）	雷达 / 饼 / 柱 / 折线 / 散点 / 力导向 graph
Diff	Monaco Editor（diff 库 CSS 回退）	Clone 回答对比，双方案容错
终端	xterm.js	Agent 执行日志回放（GitHub dark 配色）
知识图谱	**纯 Canvas 手写力导向**	RuleGraph，排斥力 + 边引力 + 中心引力 + 阻尼，无外部图库

7.3 各页面完成度

完整深度对接（核心业务）

- **Tasks**（场景 Tab + 任务列表/详情/创建）、**Dashboard**（指标卡 + 4 图表）、**Resources**（8 子 Tab CRUD）
- **Questions**（题库 + AI 出题 + 基线导入）、**Standards**（规则集 + 知识图谱 + 反哺）
- **Clone**（克隆对比 + 雷达叠加 + ROI 表）、**Regression**（版本 diff + 维度柱图）
- **CreateAgent**（AI 创造）、**Settings**（双模型库 + 探活）

可用但较轻

Pipeline / Arena / RedTeam / Observability / FineTune / Workflow / Cron / Leaderboard / Terminal / Proxy（单表 CRUD + 触发）。

报告渲染（TaskReport.vue，639 行，最复杂）

解析 result JSON，渲染总分卡 + 维度雷达柱图 + 断路器费用 + 沙箱指标 + 混沌表 + 反泄漏 + 圆桌详情 + 人工反馈。

实时运行（TaskRun.vue）

WebSocket 优先（失败回退 1.5s 轮询），三种视图：时间线 / 拓扑图 / 终端。

> **局部 mock 痕迹：**Resources 的 Agent 调试 Drawer（setTimeout 假回复）、CreateAgent 状态条（伪实时 setInterval）、Settings 通知开关（本地状态）。

八、部署与工程化

- **Docker**：多阶段构建（builder 编译 + 运行镜像），docker-compose 一键部署，healthcheck（wget `/api/health`），volume 持久化数据
- **沙箱镜像**：sandbox-go / python / node 三个独立 Dockerfile
- **CI**（GitHub Actions）：4 job 并行——backend（build + test + coverage）/ frontend（typecheck + build）/ lint（golangci-lint）/ docker（依赖前两个），用 gha cache
- **Makefile**：完整目标（build/test/test-race/coverage/vet/frontend/docker/dev），Windows/Linux 双适配
- **Nginx**：deploy/nginx.conf 提供生产反代配置

九、诚实评估：真实 / 占位 / Bug

9.1 真实且扎实的部分（平台核心价值）

1. **评测引擎主流程**（engine.go 8 步管线）——端到端跑通，含 WebSocket 实时推送
2. **圆桌合议**——并发专家 + 超时保护 + 矛盾检测，设计精良
3. **沙箱双轨执行**——Docker 池 + 进程级回退，三语言编译测试
4. **harness 六层**——ReAct 循环 + 内置工具 + MCP + Skills + Subagents + Hooks，借鉴 claw-code 落地
5. **权限门控**——三级 + 危险命令拦截 + 代码预检，安全设施完整
6. **断路器**——钱包保护，三项上限
7. **归因闭环**——根因分析 + 自动反哺 Prompt 模板
8. **前端全链路**——18 页面对接真实 API，专业可视化
9. **MITM 代理**——HTTP + HTTPS 完整实现（含 SNI 解析 + 动态证书签发）
10. **44 张表的数据库设计**——模型职责分离（考生/考官）做得规范

9.2 有降级 / 占位成分（诚实标注）

1. **RAG embedding**——hashing-trick 占位，非真实 embedding 模型
2. **无 Key 时大量 mock**——判官/出题/Agent 回答/归因均有 mock 兜底（设计决策，保证 Demo 可演示，属合理降级）
3. **Cost 估算**——硬编码定价表，非真实计费
4. **manual 规则打分**——占位（阶段①不做人工 UI）
5. **协议逆向激进能力**——TLS 指纹 / IPC 劫持等 `return ErrNotImplemented`（诚实降级，合规考虑）
6. **加密默认 key**——secrets.go 默认 key 硬编码，未接入实际路径
7. **前端局部 mock**——Agent 调试 / 创造状态条 / 通知开关

9.3 发现的真实 Bug（代码层面）

#	Bug	位置	说明
1	Arena 排行榜评分硬编码	`expansion_arena.go`	调了 ChatAgent 却丢弃结果，分数写死 75/80/70，成本恒 0.01
2	Prompt 模板版本归档 SQL 参数不匹配	`resource2.go`	3 占位符传 2 参数，版本历史静默失败
3	requestID 生成确定性	`router.go`	所有请求同一 ID，追踪失效
4	HTTP 代理 session 分组错误	`proxy.go`	所有请求进同一 session

#	Bug	位置	说明
5	Cron 解析残缺	`cron.go`	忽略日/月/周字段，不支持范围/列表
6	优先级队列是摆设	`scheduler.go`	channel FIFO，priority 未生效
7	微调导出 JOIN 语义错误	`expansion_finetune.go`	`q.scene=COALESCE(?, task_id)` 逻辑不通
8	工作流 DAG 未拓扑排序	`expansion_workflow.go`	节点顺序错则下游拿不到上游结果
9	MCP SQL 字符串拼接	`mcp_client.go:442`	参数来自 LLM，注入风险
10	migrateAddColumn 吞所有错误	`schema_core.go`	掩盖真实迁移失败
11	SeedIfEmpty 仅判 agents 表	`seed.go`	中途失败重启会跳过 seed，平台半残
12	backpressure 回路未接入主循环	`backpressure.go`	VerifyWithRetry 孤立
13	SDK 两份副本	`internal/sdk` vs `pkg/sdk`	包路径矛盾，需统一

十、总体结论

> 这是一个**完成度相当高、架构清晰、核心功能真实落地**的 AI 评测平台。它不是 PPT 项目或占位壳——编译产物存在、数据库在运行、187 个测试、145 个 API 端点、44 张表、18 个前端页面全部对接真实后端。

最扎实的部分是评测引擎（圆桌合议 + 沙箱 + 断路器 + 归因闭环）和 harness 六层能力（ReAct + Tools + MCP + Skills + Subagents + Hooks），这两块是真正的工程价值所在。**最薄弱的部分**是六大扩展方向（Arena/Finetune/Workflow 有真实 bug）和安全（无鉴权 + API Key 明文 + 加密默认 key）。

项目对自己的能力边界很诚实——`docs/ARCHITECTURE.md` 主动校准了对 claw-code 和 claude-code-router 的误读，把法律灰色能力降级为预留接口。这种"能让系统真正可用、可维护、可演示的做；灰色、高成本、低实用性的降级"的工程判断，是成熟的表现。

优先级建议

优先级	建议	理由
P0	补鉴权中间件	安全红线：终端执行/代理控制/API Key 写入全部裸奔
P1	修复 Arena / Finetune / Workflow 三个 bug	六大扩展方向中三个产出错误数据
P1	把 RAG embedding 换成真实模型	当前 hashing-trick 占位影响检索质量
P2	补 Cron 语法 + 优先级队列	调度能力目前名不副实
P2	SDK 包路径统一	internal/sdk 与 pkg/sdk 两份副本
P3	API Key 加密接入实际路径	secrets.go 设施存在但未用

> **报告说明：**本报告全部内容基于对 ai-eval-platform 全部源码的逐文件通读得出，未参考项目内的任何 TODO / MRD 文档。所有量化数据（代码行数、表数量、端点数、测试数）均为实际统计，所有 Bug 描述均定位到具体文件与代码位置。